微信小游戏从开发到上线变现：一条龙实操教程

> ¥19.9 · 示例品类：竖屏休闲三消，纯广告变现（IAA，无内购）
> 不含：完整商业源码、一对一答疑、备案文案模板（这些在 99 / 299 档）

这份教程帮你解决什么

做微信小游戏，写出能玩的第一版并不难，难的是之后那一串事：广告怎么挂才不被拒、怎么判断用户看完了、插屏为什么死活弹不出来、提审被打回该改哪里。官方文档把这些分散在十几个页面里，而且只讲 API，不讲取舍。

这份教程按**真正上线的顺序**把它们串起来。内容整理自一款已开通流量主、接入了激励视频、插屏、原生横幅三类广告的真实三消项目，教程里的频控数值、发奖规则、错误码处理，都是这个项目实际在用的配置，并附上了为什么这样配。

读完你应该能做到：

1. 知道该装哪些工具（Node、微信开发者工具、Cursor 或网页 AI 任选其一），并按固定流程改代码 → 构建 → 编译预览
2. 从零搭出一个能在开发者工具里运行的三消小游戏：工程文件、构建脚本、画布与触摸、三消核心算法、存档
3. 知道从注册到发布每一步做完是什么状态，卡住时知道自己卡在哪一步
4. 自己决定广告挂在哪、弹多频繁、什么情况发奖，并且说得出理由
5. 提审前用一张清单自查，少被打回几次

0. 全局路线图

阶段	做完的标志	最容易卡住的地方
账号	拿到 AppID，开发者工具能扫码登录	名称起得随意，后面备案名对不上
工程	开发者工具里能玩完一关	主包超限、白屏
玩法	有胜负、有关卡进度	一开始就做太多系统
广告	真机上激励能发奖、插屏能弹	只在开发者工具里测，以为广告坏了
备案	备案通过	截图跨模块重复、名称不一致
提审	审核通过	广告诱导、不看广告玩不下去
运营	流量主后台有收益数据	插屏太密，收入涨了留存掉了

顺序建议：账号和备案材料可以和开发并行。流量主有用户量门槛（以开通页为准），新游戏往往要先上线攒一批用户才能开通，所以**广告代码要先写好、用占位 ID 跑通**，开通后只换配置、不改代码。

---

1. 先定变现结构，再写玩法

纯广告游戏的收入取决于两件事：玩家愿意主动看多少次激励视频，以及你能在不伤留存的前提下展示多少次插屏。所以广告位要在设计玩法时就留好，而不是做完再找地方塞。

本示例项目的广告结构：

广告	出现场景	玩家得到什么	为什么放这里
激励视频	失败时	+5 步继续	玩家差一点就赢，此时看广告的意愿最高
激励视频	道具用完时	立刻补一个	主动需求，不打扰
激励视频	通关奖励关	+5 秒	赢了心情好，愿意多拿
插屏	通关结算	无（曝光）	一局结束的天然停顿点
插屏	进入大厅后	无（曝光）	每次打开都有一次曝光
原生模板横幅	只在大厅	无（常驻曝光）	大厅不需要操作区，局内绝不放

一条原则：**激励视频要给玩家明确、可预期的好处**（+5 步、+5 秒），不要「看广告抽随机大奖」。前者转化稳定，审核也省心。

---

2. 账号与资质

1. 在微信公众平台注册小游戏，记下 AppID。
2. **名称一次想好**：它要和后面的备案名、游戏内标题三处保持一致，改起来很麻烦。
3. 选游戏类目，按页面提示准备材料（常见有软件著作权、游戏自审自查材料等；有内购的游戏还涉及版号，纯广告游戏的要求以平台当时规则为准）。
4. 安装微信开发者工具，在公众平台「成员管理」里把开发者加进来。
5. 了解流量主入口：公众平台 → 流量主。小游戏广告走的就是这里，**不需要另外接第三方广告联盟 SDK**。

2.5 纯小白：开发工具怎么选（含 AI 辅助）

如果你从来没写过代码，先别被「引擎、框架、SDK」吓到。做本教程这类 **Canvas + TypeScript 微信小游戏**，你只需要下面两类工具：**一类是微信官方必装的，一类是写代码用的（可以用 AI 帮你写）**。

必装（没有替代品）

工具	干什么	去哪下
Node.js（建议 LTS 版）	安装依赖、把 TypeScript 打包成 `game.js`	nodejs.org
微信开发者工具	预览小游戏、上传提审、真机预览	微信开放文档 · 开发者工具

装完 Node 后，在终端里输入 `node -v`，能显示版本号就说明装对了。微信开发者工具用管理员微信扫码登录，导入项目时填你的 AppID（测试可以先选测试号，但**测广告必须用正式 AppID**）。

写代码用什么（三选一即可）

方案	适合谁	说明
Cursor（推荐）	完全零基础、想少查文档	基于 VS Code 的编辑器，自带 AI 对话和改代码。打开项目文件夹，用中文描述「帮我写一个 8×8 三消的交换逻辑」，它会直接改文件。
VS Code + 任意 AI 插件	已有 VS Code 习惯	和 Cursor 类似，只是 AI 插件要自己装（如 Copilot、通义、Codeium 等）。
网页 ChatGPT / Claude / 国内大模型	不想装新软件	把教程里的代码片段和报错复制到对话框，让 AI 解释或改一版。你要自己复制回项目里保存。

你提到的 **Codex**：现在多指 OpenAI 体系里的编程能力（常出现在 ChatGPT 或 API 里），和 Cursor 里的模型一样，**都是辅助写代码，不能代替微信开发者工具**。选 Cursor 或选「网页 Claude / ChatGPT」二选一即可，不必全装。

不需要为了做本教程去装 Cocos / Unity。 那些是游戏引擎，学习曲线更长；本教程走的是「轻量 Canvas + 自己写逻辑」，更适合个人和小团队快速上线 IAA。

AI 能帮你什么、不能帮你什么

可以交给 AI	不要完全交给 AI
按教程生成 `game.js` 最小示例、三消算法、目录结构	替你注册 AppID、开通流量主、点「提交审核」
解释报错、改 TypeScript 类型、写 `packOptions.ignore`	编造 `adunit-` 广告位 ID（必须去流量主后台复制）
把本章教程改写成「关卡配置表」「广告配置文件」	保证一次生成就无 bug（一定要自己在开发者工具里编译、真机测广告）

推荐工作流（每天重复这几步）

```
1. 用 Cursor（或 VS Code）改 src/ 里的代码
2. 终端运行：npm run build（或 npm run dev 监听修改）
3. 打开微信开发者工具 → 点「编译」→ 看模拟器
4. 广告相关：必须「预览 → 扫码真机」再测
5. 卡住时：把报错原文 + 你刚改的文件名发给 AI，不要只发截图
```

给 AI 的第一条提示词（可直接复制）

> 我要做微信小游戏，竖屏 Canvas，不用 Cocos。请按以下约束生成 TypeScript：
>
> - 入口打包为根目录 game.js
> - 玩法：8×8 三消，非法交换不扣步
> - 广告：激励复活、结算插屏、大厅原生模板横幅，广告 API 只放在一个 adapter 文件里
> - 请先列出目录结构，再写最小可运行代码

之后每次加功能，用「在当前项目基础上增加 XXX，不要改广告发奖逻辑」这种**增量描述**，比从零重写更稳。

安全提醒（发给 AI 之前）

- **不要**把 AppID、流量主 `adunit-`、云开发 envId、备案主体信息贴给 AI 或公开仓库。
- 教程和 AI 生成的代码里，广告位用 `adunit-YOUR_xxx` 占位，上线前自己替换。

---

3. 工程组织

3.1 推荐分层

最重要的一条规则：**只有一个地方允许调用 `wx` 的广告接口。**

```
config/          关卡表、广告位 ID、频控数值
logic/           玩法规则，不碰 wx
core/adapters/   只在这里调用 wx 广告 / 存储 / 音频
presentation/    界面和渲染
```

这样做的回报很直接：换广告位 ID 只改配置文件；调插屏频率只改一个数字；微信接口变了只改一个文件；玩法逻辑能在电脑上直接跑单元测试，不用每次都开模拟器。

3.2 最小工程：三个文件就能跑

微信小游戏不需要游戏引擎也能做。示例项目就是用原生 Canvas 自己画的，包体最小。一个能在开发者工具里运行的小游戏，根目录至少要有这三个文件：

```
my-game/
├── game.json              小游戏配置
├── project.config.json    开发者工具的工程配置
└── game.js                入口代码
```

`game.json`：

```
{
  "deviceOrientation": "portrait",
  "showStatusBar": false
}
```

`project.config.json`（`appid` 换成你自己的）：

```
{
  "compileType": "game",
  "appid": "你的AppID",
  "projectname": "my-game",
  "packOptions": {
    "ignore": [
      { "value": "node_modules", "type": "folder" },
      { "value": "src", "type": "folder" },
      { "value": "scripts", "type": "folder" },
      { "value": "package.json", "type": "file" },
      { "value": "package-lock.json", "type": "file" }
    ]
  }
}
```

`game.js` 先写一个最简单的版本，确认能画出东西、能响应触摸：

```
const canvas = wx.createCanvas(); // 第一次调用得到的就是屏幕画布
const ctx = canvas.getContext('2d');
const info = wx.getSystemInfoSync();
canvas.width = info.windowWidth * info.pixelRatio;
canvas.height = info.windowHeight * info.pixelRatio;
ctx.scale(info.pixelRatio, info.pixelRatio); // 之后都按逻辑像素画，高清屏不会糊

let tapX = -1;
let tapY = -1;
wx.onTouchStart((e) => {
  tapX = e.touches[0].clientX;
  tapY = e.touches[0].clientY;
});

function frame() {
  ctx.fillStyle = '#ffe9d6';
  ctx.fillRect(0, 0, info.windowWidth, info.windowHeight);
  ctx.fillStyle = '#ff7a45';
  ctx.font = '24px sans-serif';
  ctx.fillText('Hello 小游戏', 40, 120);
  if (tapX >= 0) {
    ctx.beginPath();
    ctx.arc(tapX, tapY, 20, 0, Math.PI * 2);
    ctx.fill();
  }
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

在开发者工具里选「小游戏 → 导入项目」，目录选 `my-game`，看到文字、点屏幕出现圆点，工程就通了。

3.3 用 TypeScript 写，打包成 game.js

游戏逻辑变复杂以后，建议用 TypeScript 写在 `src/` 里，再打包成一个 `game.js`。示例项目用的是 esbuild，速度快、配置少：

```
npm init -y
npm i -D esbuild typescript
```

新建 `scripts/build.mjs`：

```
import * as esbuild from 'esbuild';

const watch = process.argv.includes('--watch');
const options = {
  entryPoints: ['src/main.ts'],
  bundle: true,
  outfile: 'game.js',
  format: 'iife',
  platform: 'neutral',
  target: ['es2018'],
  sourcemap: true,
};

if (watch) {
  const ctx = await esbuild.context(options);
  await ctx.watch();
} else {
  await esbuild.build(options);
}
```

在 `package.json` 里加上 `"type": "module"`，以及：

```
"scripts": {
  "build": "node scripts/build.mjs",
  "dev": "node scripts/build.mjs --watch"
}
```

开发时运行 `npm run dev`，改完代码在开发者工具里点「编译」就能看到效果。TypeScript 里用到 `wx` 会报类型错误，可以安装社区维护的小游戏类型声明包，或者先写一句 `declare const wx: any;` 应急。

3.4 控制上传体积

上面 `packOptions.ignore` 的作用是让上传时跳过源码和依赖，只带 `game.js` 和资源文件。很多人第一次上传失败，就是因为把 `node_modules` 打进了包。另外记得把 `game.js.map` 也加进忽略列表。主包体积上限以微信官方文档为准（常见口径为 4MB），图片和音效多的话放分包或 CDN。

---

4. 制作玩法：以三消为例

按这个顺序做，每一步结束都有能玩的东西。下面的代码是能直接用的核心算法，界面和动画按你的美术风格自己画。

4.1 棋盘数据

用二维数组存每格的方块种类，`0` 表示空：

```
const ROWS = 8;
const COLS = 8;
const KINDS = 5; // 5 种颜色的方块，编号 1～5
const EMPTY = 0;

const randomKind = () => 1 + Math.floor(Math.random() * KINDS);

/** 生成开局棋盘：每填一格都避开「左边两格同色」和「上面两格同色」，保证开局没有现成的三连 */
function createBoard() {
  const board = [];
  for (let r = 0; r < ROWS; r++) {
    board.push([]);
    for (let c = 0; c < COLS; c++) {
      let kind;
      do {
        kind = randomKind();
      } while (
        (c >= 2 && board[r][c - 1] === kind && board[r][c - 2] === kind) ||
        (r >= 2 && board[r - 1][c] === kind && board[r - 2][c] === kind)
      );
      board[r].push(kind);
    }
  }
  return board;
}
```

4.2 找出所有三连

横着扫一遍、竖着扫一遍，连续 3 个及以上同色的格子都记下来：

```
function findMatches(board) {
  const hit = new Set(); // 存 r * COLS + c

  for (let r = 0; r < ROWS; r++) {
    let run = 1;
    for (let c = 1; c <= COLS; c++) {
      const same = c < COLS && board[r][c] !== EMPTY && board[r][c] === board[r][c - 1];
      if (same) {
        run++;
      } else {
        if (run >= 3) for (let k = c - run; k < c; k++) hit.add(r * COLS + k);
        run = 1;
      }
    }
  }

  for (let c = 0; c < COLS; c++) {
    let run = 1;
    for (let r = 1; r <= ROWS; r++) {
      const same = r < ROWS && board[r][c] !== EMPTY && board[r][c] === board[r - 1][c];
      if (same) {
        run++;
      } else {
        if (run >= 3) for (let k = r - run; k < r; k++) hit.add(k * COLS + c);
        run = 1;
      }
    }
  }

  return hit;
}
```

4.3 消除、下落、补新块、连锁

```
/** 每一列从下往上把方块压实，顶部空出来的格子补新块 */
function collapse(board) {
  for (let c = 0; c < COLS; c++) {
    let write = ROWS - 1;
    for (let r = ROWS - 1; r >= 0; r--) {
      if (board[r][c] !== EMPTY) {
        board[write][c] = board[r][c];
        if (write !== r) board[r][c] = EMPTY;
        write--;
      }
    }
    for (let r = write; r >= 0; r--) board[r][c] = randomKind();
  }
}

/** 反复「找三连 → 消除 → 下落」，直到没有新的三连。返回一共消了多少格 */
function resolve(board) {
  let cleared = 0;
  for (;;) {
    const hit = findMatches(board);
    if (hit.size === 0) return cleared;
    hit.forEach((i) => { board[Math.floor(i / COLS)][i % COLS] = EMPTY; });
    cleared += hit.size;
    collapse(board);
  }
}
```

4.4 交换与步数

只允许上下左右相邻交换。交换后没有三连就换回来，**不扣步数**，这样玩家试错不会有挫败感：

```
const state = { board: createBoard(), movesLeft: 20, score: 0 };

function trySwap(r1, c1, r2, c2) {
  if (Math.abs(r1 - r2) + Math.abs(c1 - c2) !== 1) return false;
  const b = state.board;
  [b[r1][c1], b[r2][c2]] = [b[r2][c2], b[r1][c1]];
  if (findMatches(b).size === 0) {
    [b[r1][c1], b[r2][c2]] = [b[r2][c2], b[r1][c1]]; // 没有三连，换回去
    return false;
  }
  state.movesLeft--;
  state.score += resolve(b) * 10; // 每消一格 10 分
  return true;
}
```

把触摸坐标换算成格子：

```
const cell = Math.floor(Math.min(info.windowWidth, info.windowHeight) * 0.9 / COLS);
const boardLeft = (info.windowWidth - cell * COLS) / 2;
const boardTop = 160;

function hitCell(x, y) {
  const c = Math.floor((x - boardLeft) / cell);
  const r = Math.floor((y - boardTop) / cell);
  return r >= 0 && r < ROWS && c >= 0 && c < COLS ? { r, c } : null;
}
```

交互可以先做最简单的「点一格选中，再点相邻一格交换」，跑通以后再加滑动手势和动画。

4.5 关卡与胜负

关卡写成配置表，方便以后调难度不改代码：

```
const LEVELS = [
  { id: 1, moves: 15, targetScore: 600 },
  { id: 2, moves: 15, targetScore: 900 },
  { id: 3, moves: 18, targetScore: 1300 },
];
```

每次交换后判断：分数达到目标就通关；步数用完还没达到就失败。失败时就是第 5 节「激励视频复活」的挂载点。

4.6 保存进度

```
function saveProgress(maxLevel) {
  wx.setStorageSync('progress', { maxLevel });
}

function loadProgress() {
  return wx.getStorageSync('progress') || { maxLevel: 1 };
}
```

本地存储足够首版使用，云存档放到以后的版本。

4.7 后续顺序

核心玩法跑通后，按这个顺序继续：

1. **手感**：交换、消除、下落的动画，消除音效。三消好不好玩，一大半在这里。
2. **内容**：更多关卡、收集指定颜色等目标、障碍物、四连和五连生成特殊方块。
3. **变现挂钩**：失败复活、道具补给、结算插屏、大厅横幅（先用占位 ID，见第 5 节）。
4. **上架必备**：图标、分享图、隐私授权弹窗、加载页。

核心算法建议写单元测试（这部分不依赖微信，在电脑上用 Node 就能跑），因为三消的 bug 在真机上最难复现。邀请、排行榜、云存档这类社交功能放到第二个版本。首版目标只有一个：**稳定可玩，广告能正常出。**

---

5. 挂广告：流量主详细步骤

5.1 创建广告位

开通流量主后，创建以下广告位并复制每个 `adunit-…` ID：

- 激励视频 × 1～2（复活和加时可以共用，也可以分开以便单独看数据）
- 插屏 × 1（结算和启动可共用）
- 原生模板 × 1（大厅横幅）

写进配置文件（下面是示例，换成你自己的 ID）：

```
{
  "rewarded_revive":       { "adUnitId": "adunit-YOUR_REWARD_1" },
  "rewarded_crush_extend": { "adUnitId": "adunit-YOUR_REWARD_2", "extraMs": 5000 },
  "interstitial_settle":   { "adUnitId": "adunit-YOUR_INTERSTITIAL", "everyNLevels":
2, "cooldownSeconds": 60 },
  "banner_lobby":          { "adUnitId": "adunit-YOUR_CUSTOM_BANNER", "enabled": true
}
}
```

建议代码里识别占位 ID（比如以 `YOUR_` 或 `TODO_` 开头），遇到就不创建广告实例。这样开通流量主之前也能正常开发，不会被报错刷屏。

5.2 三个关键决定

决定一：什么时候算「看完了」，可以发奖?

激励视频关闭时，微信回调会带一个 `isEnded` 字段。但流量主后台可以设置激励的最长观看时长：如果你设的是 15 秒，而广告片本身有 30 秒，玩家看满 15 秒关掉时 `isEnded` 也可能是 `false`。只认 `isEnded` 的话，玩家明明看够了却拿不到奖励，投诉和差评就来了。

示例项目的做法：`isEnded` 为 `true`，或者从展示到关闭已经超过 14 秒（比后台设置的 15 秒略短），都算看完。这个阈值要**跟你在流量主后台的设置对齐**。

决定二：插屏弹多频繁?

示例项目的配置是**每 2 关弹一次、两次之间至少隔 60 秒、失败的局不弹**。失败不弹是因为玩家正处在挫败感里，这时再弹插屏，流失风险最大。

决定三：插屏为什么一启动就弹不出来?

微信对插屏有频率限制：小游戏启动后一小段时间内调用插屏会失败，错误码 2001；两次插屏间隔太短会返回 2002。所以「进大厅就弹插屏」必须这样做：

1. 先让大厅画出来，延迟 2～3 秒再调用
2. 遇到 2001 就隔几秒重试
3. 设一个总超时（示例项目是 45 秒），超时或玩家进了关卡就放弃

如果没有这套重试，你会发现启动插屏的展示率低得离谱，却查不出原因。

5.3 代码原则

- 激励视频用 `wx.createRewardedVideoAd`，插屏用 `wx.createInterstitialAd`，大厅横幅用 `wx.createCustomAd`（原生模板）。
- 每个广告位**只创建一个实例**，重复创建会让回调叠加，一次关闭触发多次发奖。

- 界面按钮只调用统一的 `show(广告位)`，返回四种结果之一：看完、跳过、出错、未就绪。**只有「看完」发奖**，其余三种提示稍后再试。
- 每次展示结束后马上预加载下一条，否则第二次点击要等加载。
- 横幅进关卡时立即隐藏。还要防一种情况：横幅的显示请求发出去了，但玩家已经进了关卡，横幅这时才加载出来。要在加载完成的回调里再检查一次「现在还该不该显示」。

5.4 一定要用真机测

开发者工具对原生模板广告支持不完整，经常报 `insertTextView` 之类的错，这不代表你的代码有问题。广告相关的验收**只以真机预览为准**。

真机上逐项确认：

- [ ] 失败后看完激励，得到奖励；看到一半关掉，没有奖励
- [ ] 道具补给、奖励关加时同上
- [ ] 结算插屏按「每 N 关、冷却 60 秒」出现，冷却期内不再弹
- [ ] 大厅有横幅，进关后横幅消失
- [ ] 关闭广告回到游戏后，画面正常、还能操作、背景音乐恢复
- [ ] 没有广告可展示时（常见错误码 1004，表示暂时没有合适的广告），提示友好，游戏不卡死

上面倒数第二条最容易被忽略：广告关闭后真机上可能黑屏、点击失灵，或者关广告那一下的点击直接穿透到棋盘上走了一步。99 档的 FAQ 里有完整的原因和处理办法。

5.5 红线

以下做法会导致审核被拒，或流量主被限流、封禁：

- 诱导点击广告，或设计假的关闭按钮
- 不看广告就玩不下去主线
- 激励视频的奖励文案和实际发放的不一致

---

6. 备案与合规要点

1. 按微信后台指引完成小游戏备案。
2. **备案名 = 公众平台名称 = 游戏内标题**，三处一字不差。
3. 纯广告游戏的系统功能描述里不要出现商城、虚拟货币、VIP。
4. 场景、玩法、系统三个模块的截图**不能重复使用同一张**，这是被打回最多的原因之一。
5. 配置用户隐私保护指引，并在需要时弹出授权。

7. 提审与发布

7.1 发布前检查

- [ ] 用正式 AppID 构建了最新版本
- [ ] 配置里没有残留 `YOUR_` / `TODO_` 占位广告 ID
- [ ] 广告在真机上验收通过
- [ ] 图标、简介、分享图齐全
- [ ] 隐私指引已配置
- [ ] 备案状态满足平台要求
- [ ] 主包体积合格，没有把无关文件打进包

7.2 流程

1. 开发者工具点「上传」，版本备注写清楚改了什么（以后回溯用得到）。
2. 公众平台 → 版本管理 → 提交审核。
3. 审核说明里写明玩法路径和广告用途，例如：「激励视频用于失败复活和道具补给，插屏在通关结算时展示，不看广告也可以重试或返回大厅。」
4. 审核通过后发布。如果有灰度发布选项，可以先放一部分用户观察。
5. 发布后立即用正式版扫码，把广告和存档再走一遍。

8. 上线后怎么调

频率	看什么	怎么调
每天	流量主收益是否异常、有没有崩溃反馈	收益突然归零，先查广告位状态
每周	收入和次日留存放在一起看	收入涨但留存掉，就把插屏改成每 3 关
每个版本	卡关最严重的几关	先降难度，而不是先加广告

数值都放在配置文件里，调整时不用改代码。

9. 常见问题速查

现象	先查这里
白屏	入口文件没有重新构建，或者启动时就抛了错（真机打开调试看日志）
改了代码没变化	没有重新构建，或者开发者工具没重新编译
开发者工具里广告报错	正常现象，换真机测
激励看完不发奖	完成判定只认了 `isEnded`，见 5.2 决定一
启动插屏几乎不出	没处理 2001 频控，见 5.2 决定三
游客 AppID 没有广告	需要正式 AppID 并开通流量主
审核因广告被拒	去掉诱导文案，保证不看广告也能继续
审核因名称被拒	统一备案名、平台名称、游戏内标题

下一档包含什么

- **¥99 完整包**：本教程，加上可以直接复制的广告配置模板、提审和广告验收表、备案文案模板（场景/玩法/系统三模块），以及一份按「现象 → 原因 → 处理」整理的 FAQ，覆盖广告错误码对照、关广告后黑屏、点击穿透等真机问题。
- **¥299 旗舰包**：99 档全部内容，加上分层架构与广告策略设计、玩法和变现结合的设计提纲、可改造使用的广告适配和前后台处理代码片段，以及 1 次文字答疑。

平台规则会调整，开通门槛、包体上限、备案要求以微信官方最新说明为准。